

IMAGO Media Search Challenge

Next.js, TypeScript, Tailwind, lightweight indexed search

Architecture overview

The application is a Next.js TypeScript app with three server endpoints: `/api/search` for paginated search, `/api/facets` for filter values, and `/api/analytics` for demo usage metrics. A module-level search service creates a deterministic 12,000 item IMAGO-like dataset, preprocesses it once, and builds an in-memory token map. The client UI uses debounced requests, accessible native controls, and Tailwind styling.

Search strategy and relevance

Keyword search runs across `suchtext`, `fotografen`, and `bildnummer`. Text is lowercased, de-accented, delimiter-normalized, tokenized, and filtered for common stop words. Scoring is weighted by field: `suchtext` is primary with +8 exact and +3 prefix matches, `fotografen` is secondary with +4 exact and +1.5 prefix matches, and `bildnummer` gets +2 for direct lookups. Results default to score order with date as a tie-breaker, or can be sorted by date ascending/descending.

Preprocessing strategy

Preprocessing extracts restrictions from compact caption text such as `PUBLICATIONxINxGERxSUIxAUTxONLY`, normalizing it to `PUBLICATION_IN_GER_SUI_AUT_ONLY`. Dates are parsed into ISO `YYYY-MM-DD` while preserving the original datum value. Caption, credit, and image number tokens are computed once at module load, then indexed into `Map<token, Set<itemIndex>>`. This improves request consistency and avoids repeated text parsing.

Scaling approach

For millions of records, I would move indexing to ingestion time and use Elasticsearch, OpenSearch, or Meilisearch. Media metadata changes would publish events to a queue, ingestion workers would normalize dates, restrictions, people, locations, and caption tokens, and the search engine would provide analyzers, field boosts, faceting, typo tolerance, and efficient date filtering. Analytics would move from memory to durable aggregated storage with latency percentiles and zero-result tracking.

Testing approach

Vitest covers date parsing, restriction extraction, tokenization, paginated search, combined filtering, and analytics. In a production setting I would add Playwright tests for search input debouncing, keyboard navigation, filter changes, pagination, empty states, and API contract fixtures based on real seed data.

Trade-offs

The challenge mentioned `seed.json`, but no file was present in this workspace, so I generated deterministic seed data with the required fields. Prefix matching currently scans token keys, which is acceptable for the demo scale but would be replaced by an n-gram/trie strategy or external search engine at larger scale. Snippets are plain text excerpts, analytics are in-memory, and image previews

are omitted because the schema did not include media URLs.